

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
De Béni Mellal

TP 3

Architecture Distribuée

Utilisation de micro services

Réalisé par :

BAH Thierno Mamoudou

Encadrant :

Pr. BE ELBAGHAZAOUI

Année universitaire : 2025 — 2026

1. Introduction

Ce rapport presente la realisation du TP 3 portant sur la mise en place d'une architecture distribuee complete composee de 6 composants interconnectes. L'objectif est de comprendre et implementer un systeme de gestion de commandes utilisant des technologies modernes du web.

Les objectifs principaux de ce TP sont :

- Deployer une architecture distribuee avec 6 composants interconnectes
- Comprendre le role de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages
- Implementer un flux complet : POST commande -> stockage MongoDB -> cache Redis -> notification RabbitMQ
- Tester et observer le comportement distribue avec Postman

2. Architecture du Système

2.1 Présentation Générale

L'architecture mise en place est composée des 6 éléments suivants :

Composant	Technologie	Role
Client	Postman	Envoie les requetes HTTP POST/GET pour creer et lister des commandes
Load Balancer	Nginx	Recoit le trafic et le repartit entre 2 instances Node.js (port 3001 et 3002)
Serveur applicatif	Node.js / Express	Implements la logique metier : validation, traitement et reponse
Base de donnees	MongoDB	Stocke toutes les commandes de facon persistante
Cache	Redis	Memorise les resultats GET pour eviter des requetes repetitives a MongoDB
Bus de messages	RabbitMQ	Recoit un evenement apres chaque commande et affiche une notification

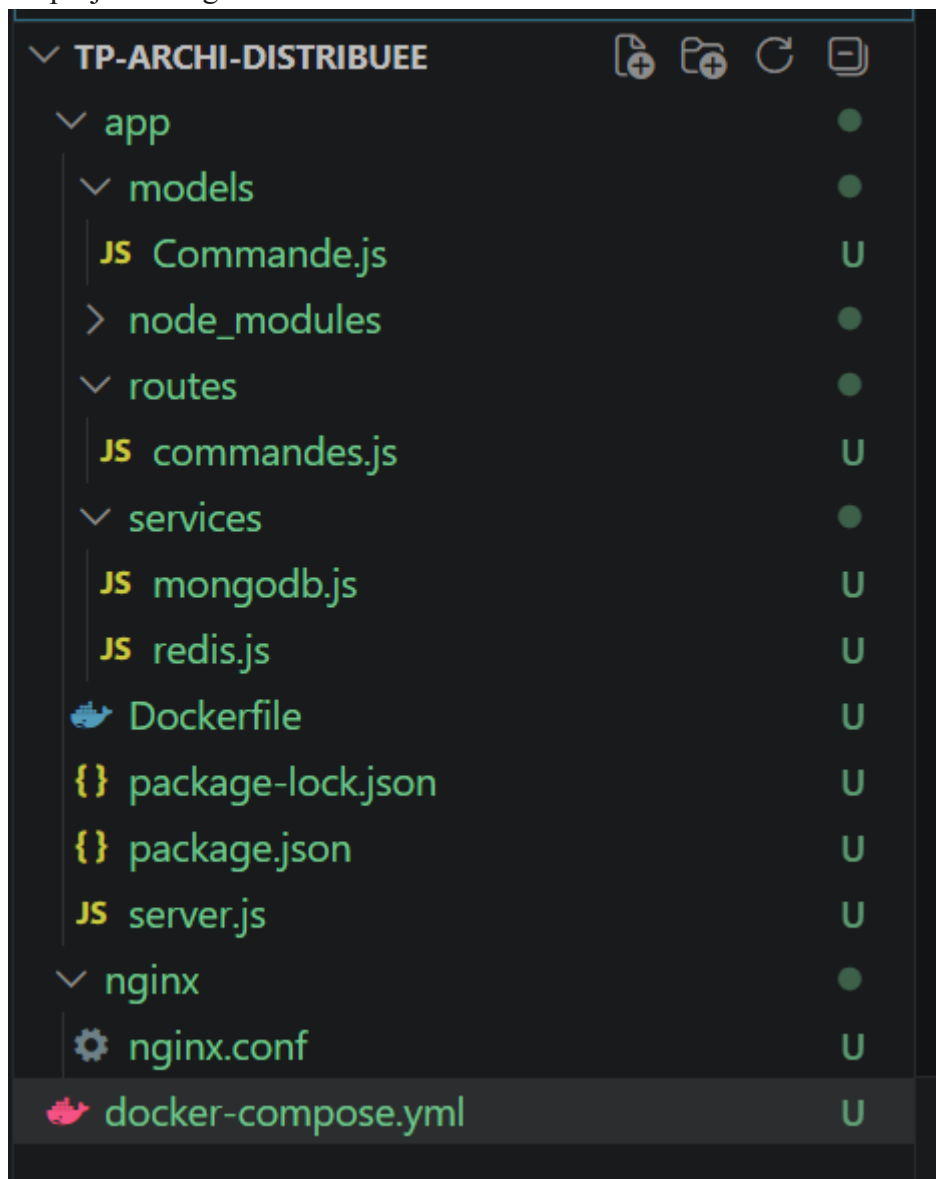
2.2 Flux de Donnees

Le flux de donnees dans le systeme suit le chemin suivant :

- Postman -> Nginx :80 -> Node.js :3001 / :3002
- Node.js -> MongoDB (stockage persistant)
- Node.js -> Redis (cache GET /commandes, TTL = 30 secondes)
- Node.js -> RabbitMQ -> Consommateur -> Notification console

3. Structure du Projet

Le projet est organisé selon la structure suivante :



4. Implémentations

4.1 Docker Compose

Le fichier docker-compose.yml orchestre le démarrage de tous les services. Il définit 6 services : mongodb, redis, rabbitmq, app1, app2, et nginx. Le mot-cle depends_on garantit le bon ordre de démarrage des conteneurs.

Les deux instances Node.js (app1 et app2) sont des copies identiques du même serveur. Elles se distinguent uniquement par leurs variables d'environnement PORT et INSTANCE, ce qui permet d'identifier quelle instance traite chaque requête.

4.2 Configuration Nginx - Load Balancer

Nginx est configure comme load balancer avec l'algorithme Round-robin par default. Il ecoute sur le port 80 et repartit les requetes entre app1:3001 et app2:3002 en alternance.

```
http {  
    upstream backend {  
        server app1:3001;  
        server app2:3002;  
    }  
}
```

4.3 Services Node.js

Service MongoDB (mongodb.js)

Ce service definit le schema d'une commande avec les champs : produit, quantite, client, statut et date. Il etablit la connexion vers la base de donnees MongoDB en utilisant Mongoose comme ORM.

Service Redis (redis.js)

Ce service cree la connexion vers Redis avec ioredis. Il est utilise pour mettre en cache les resultats des requetes GET avec un TTL (Time To Live) de 30 secondes, reduisant ainsi la charge sur MongoDB.

Service RabbitMQ (rabbitmq.js)

Ce service gere la connexion vers RabbitMQ. Il expose deux fonctions : connectRabbitMQ() qui etablit la connexion et cree la file d'attente 'commandes', et publierCommande() qui envoie un message dans cette file lors de chaque nouvelle commande. Un mecanisme de retry (10 tentatives espacees de 5 secondes) a ete ajoute pour gerer le demarrage progressif des conteneurs.

4.4 Routes API

POST /commandes

Cette route cree une nouvelle commande. Elle effectue les operations suivantes dans l'ordre :

- Validation des champs obligatoires (produit, quantite, client)
- Sauvegarde de la commande dans MongoDB
- Invalidation du cache Redis (suppression de la cle 'toutes_commandes')
- Publication d'un message dans RabbitMQ
- Retour d'une reponse 201 avec les details de la commande et l'instance qui a traite la requete

GET /commandes

Cette route retourne toutes les commandes avec gestion du cache :

- Verification du cache Redis : si les donnees sont presentes, retour immediat (source: Cache Redis)
- Si le cache est vide : recuperation depuis MongoDB, stockage en cache avec TTL de 30s, retour des donnees (source: MongoDB)

4.5 Consommateur RabbitMQ (consumer.js)

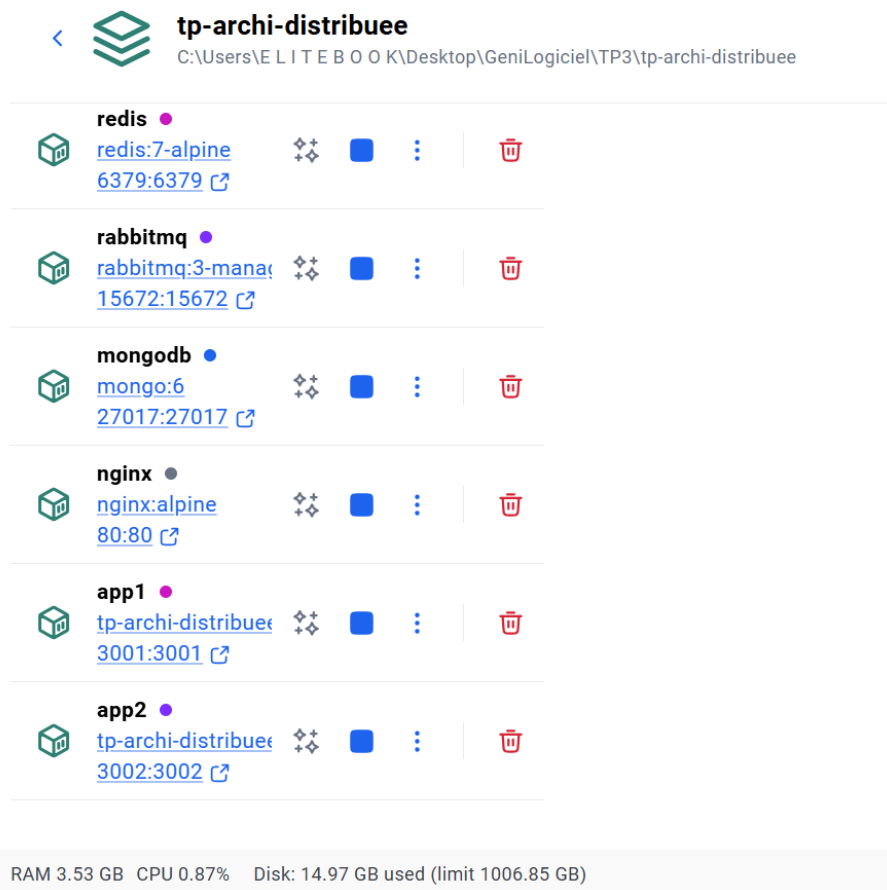
Le consommateur simule un microservice de notification independant. Il ecoute en permanence la file d'attente 'commandes' et affiche une notification dans la console pour chaque nouvelle commande recue, simulant l'envoi d'un email de confirmation.

5. Tests — Resultats

5.1 Verification du Demarrage

Après lancement de `docker-compose up --build`, tous les 6 conteneurs ont démarré avec succès. La vérification via `docker ps` confirme que `nginx`, `redis`, `rabbitmq` et `mongodb` sont actifs. Les deux instances `Node.js` (`app1` et `app2`) sont connectées aux trois services.

Les messages de démarrage observés dans les logs :



tp-archi-distribuee
C:\Users\ELITEBOK\Desktop\GeniLogiciel\TP3\tp-archi-distribuee

Container	Image	Status	Ports	Actions
redis	redis:7-alpine	Running	6379:6379	Stop, Restart, Logs, Delete
rabbitmq	rabbitmq:3-management	Running	15672:15672	Stop, Restart, Logs, Delete
mongodb	mongo:6	Running	27017:27017	Stop, Restart, Logs, Delete
nginx	nginx:alpine	Running	80:80	Stop, Restart, Logs, Delete
app1	tp-archi-distribuee	Running	3001:3001	Stop, Restart, Logs, Delete
app2	tp-archi-distribuee	Running	3002:3002	Stop, Restart, Logs, Delete

RAM 3.53 GB CPU 0.87% Disk: 14.97 GB used (limit 1006.85 GB)

5.2 Test 1 : Creation d'une Commande (POST)

Configuration de la requête dans Postman :

- Methode : POST
- URL : `http://localhost/commandes`
- Header : Content-Type: `application/json`

POST http://localhost/commandes

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "produit": "pc laptop",
3   "quantite": 3,
4   "client": "mud"
}

```

Body Cookies Headers (7) Test Results (1/1) 201 Created • 204 ms • 4

{ } JSON Preview Visualization

```

1 {
2   "message": "Commande créée",
3   "commande": {
4     "produit": "pc laptop",
5     "quantite": 3,
6     "client": "mud",
7     "statut": "en_attente",
8     "_id": "69cb4edf1e4d1c15b88dcae9",
9     "date": "2026-03-31T04:34:39.341Z",
10    "__v": 0
11  },
12  "traitee_par": "Instance-2"
}

```

Dans les logs Docker, les messages suivants ont été observés :

📬 Message public dans RabbitMQ 📬 NOUVELLE
 COMMANDE RECUE ! 📧 Email de confirmation envoyé
 (simule)

5.3 Test 2 : Observation du Cache Redis (GET)

Premier appel GET sur http://localhost/commandes :

GET http://localhost/commandes

Docs Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (7) Test Results (1/1) 200 OK • 102 ms • 450 B

{ } JSON Preview Visualize

```

1 {
2   "source": "MongoDB",
3   "traitee_par": "Instance-1",
4   "commandes": [
5     {
6       "_id": "69cb4edf1e4d1c15b88dcae9",
7       "produit": "pc laptop",
8       "quantite": 3,
9       "client": "mud",
10      "statut": "en_attente",
11      "date": "2026-03-31T04:34:39.341Z",
12      "__v": 0
13    }
14  ]
}

```

Deuxieme appel GET (dans les 30 secondes) :

GET http://localhost/commandes

Docs Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value
Key	Value

Body Cookies Headers (7) Test Results (1/1) 200 OK

{ } JSON Preview Visualize

```

1  {
2    "source": "MongoDB",
3    "traitee_par": "Instance-2",
4    "commandes": [
5      {
6        "_id": "69cb4edf1e4d1c15b88dcae9",
7        "produit": "pc laptop",
8        "quantite": 3,
9        "client": "mud",
10       "statut": "en_attente",
11       "date": "2026-03-31T04:34:39.341Z",
12       "__v": 0
13     }
14   ]

```

Observation : La deuxieme reponse est instantanee car les donnees sont servies depuis le cache Redis. Le champ 'traitee_par' peut changer entre les deux appels car Nginx repartit les requetes en Round-robin.

5.4 Test 3 : Observation du Load Balancing

Envoi de 4 requetes POST successives. Resultats observes dans le champ 'traitee_par' :

Requete	Instance traitante	Observation
Requete 1	Instance-1	Premiere instance du Round-robin
Requete 2	Instance-2	Deuxieme instance du Round-robin
Requete 3	Instance-1	Retour a la premiere instance
Requete 4	Instance-2	Retour a la deuxieme instance

Premiere instance du Round-robin :

POST

http://localhost/commandes

Docs

Params

Authorization

Headers (8)

Body

Scripts

none

form-data

x-www-form-urlencoded

raw

binary

```

1  {
2    "produit": "samsung",
3    "quantite": 7,
4    "client": "Bah"
5  }

```

Body

Cookies

Headers (7)

Test Results (1/1)

20

JSON

Preview

Visualization

```

5    "commande": {
6      "produit": "samsung",
7      "quantite": 7,
8      "client": "Bah",
9      "statut": "en_attente",
10     "_id": "69cb4fc3a6b5965f77bdb471",
11     "date": "2026-03-31T04:38:27.828Z",
12     "__v": 0
13   },
14   "traitee_par": "Instance-1"
15 }

```

Deuxieme instance du Round-robin :

POST

http://localhost/commandes

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

```

1  {
2    "produit": "samsung",
3    "quantite": 7,
4    "client": "Bah"
5  }

```

Body

Cookies

Headers (7)

Test Results (1/1)

201 Created

34

JSON

Preview

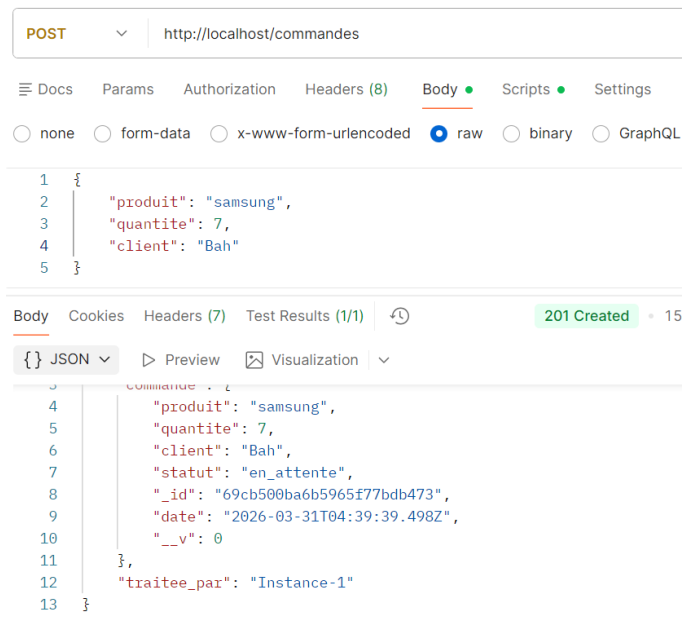
Visualization

```

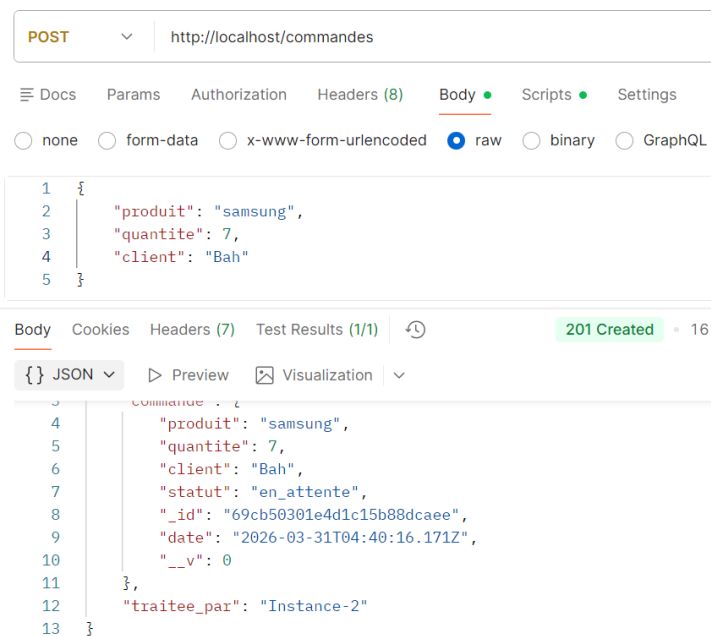
5    "commande": {
6      "produit": "samsung",
7      "quantite": 7,
8      "client": "Bah",
9      "statut": "en_attente",
10     "_id": "69cb4ffc1e4d1c15b88dcaec",
11     "date": "2026-03-31T04:39:24.777Z",
12     "__v": 0
13   },
14   "traitee_par": "Instance-2"
15 }

```

Retour a la premiere instance :



Retour a la deuxieme instance :



Cette alternance confirme que Nginx repartit la charge en Round-robin entre les deux instances.

6. Réponses aux questions

Q1 : Quel composant recoit en premier les requetes ? Quel algorithme utilise-t-il ?

Le premier composant a recevoir les requetes du client est Nginx, qui joue le role de Load Balancer. Il ecoute sur le port 80 et redistribue les requetes entre les deux instances Node.js (app1 et app2).

L'algorithme utilise est le Round-robin : les requetes sont distribuees en alternance, une requete vers Instance-1, la suivante vers Instance-2, puis Instance-1, etc. Cela permet de repartir la charge de facon equilibree entre les deux serveurs.

Q2 : Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il a fonctionner ?

Si app1 tombe en panne, Nginx detecte automatiquement qu'elle ne repond plus et redirige toutes les requetes vers app2 uniquement. Le systeme continue donc de fonctionner normalement sans interruption de service.

Nginx continue de fonctionner car il est concu pour gerer les pannes des serveurs. C'est justement l'un des roles principaux d'un Load Balancer : assurer la continuite du service meme en cas de panne d'un serveur. C'est ce qu'on appelle la tolerance aux pannes (fault tolerance).

Q3 : Expliquez la difference entre le 1er GET (source MongoDB) et le 2eme GET (source Redis)

Lors du 1er appel GET, aucune donnee n'est encore en cache. L'instance Node.js va donc chercher les commandes directement dans MongoDB, ce qui prend un peu plus de temps. Elle stocke ensuite le resultat dans Redis avec une duree de vie (TTL) de 30 secondes.

Lors du 2eme appel GET, l'instance verifie d'abord Redis avant d'aller dans MongoDB. Comme le resultat est deja en cache, elle le retourne instantanement sans solliciter MongoDB. C'est pourquoi on voit source: Cache Redis dans la reponse.

L'avantage du cache est double : la reponse est plus rapide et MongoDB est moins sollicite, ce qui ameliore les performances globales du systeme.

Q4 : Pourquoi invalide-t-on le cache Redis apres un POST /commande ?

On invalide le cache Redis apres chaque POST car la liste des commandes a change. Si on garde l'ancien cache, le prochain GET retournerait une liste obsolete qui ne contiendrait pas la nouvelle commande.

En supprimant le cache apres chaque POST, on force le prochain GET a aller chercher la liste a jour dans MongoDB, puis a la remettre en cache avec les donnees correctes. Cela garantit la coherence des donnees entre MongoDB et Redis.

Q5 : En quoi le bus de messages RabbitMQ ameliore-t-il la resilience du systeme ?

RabbitMQ ameliore la resilience car il decouple les taches. Quand une commande est creee, le serveur n'attend pas que la notification soit envoyee pour repondre au client. Il depose simplement un message dans la file d'attente et repond immediatement.

Si le service de notification tombe en panne, les messages restent stockes dans la file et seront traites des que le service redemarre. Cela evite de perdre des notifications et n'impacte pas le reste du systeme. Sans RabbitMQ, une panne du service de notification entrainerait l'echec de toute la creation de commande.

Q6 : Si on voulait ajouter une 3eme instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?

Pour ajouter une 3eme instance, il suffit d'ajouter une ligne dans le bloc upstream backend du fichier nginx.conf :

```
upstream backend {    server app1:3001;
server app2:3002;
    server app3:3003; <- ligne a ajouter
}
```

Il faudrait egalement ajouter le service app3 dans le fichier docker-compose.yml de la meme facon que app1 et app2, avec le port 3003. Nginx integrerait automatiquement cette nouvelle instance dans son algorithme Round-robin.

7. Conclusion

Ce TP a permis de mettre en place et de comprendre une architecture distribuee complete composee de 6 composants interconnectes. Nous avons deploye avec succes un systeme de gestion de commandes integrant :

- Un load balancer Nginx assurant la repartition de charge en Round-robin
- Deux instances Node.js/Express implementant la logique metier
- Une base de donnees MongoDB pour le stockage persistant
- Un cache Redis reduisant la charge sur MongoDB avec un TTL de 30 secondes
- Un bus de messages RabbitMQ assurant le decouplage et la resilience des notifications

Les tests realises avec Postman ont confirme le bon fonctionnement de tous les composants : la creation de commandes, la gestion du cache, le load balancing Round-robin, et les notifications via RabbitMQ.

Ce TP illustre les avantages des architectures distribuees en termes de performance (cache), de scalabilite (load balancing), de resilience (bus de messages) et de disponibilite (instances multiples). Ces concepts sont fondamentaux dans le developpement de systemes modernes a grande echelle.